PROGRAMLANABİLİR DENETLEYİCİLER

DERS – 06

Temel Programlama Komutları

2.8. Temel Programlama Komutları

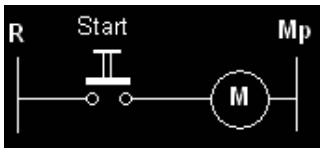
2.8.1. Start, Stop ve Çıkışlar

Klasik kumanda devrelerinde sistemin çalışmasını başlatmak ve durdurmak için kullanılan start ve stop butonları ile kumanda akışını yönlendiren sensör, sınır anahtarı vb. elemanlar PLC cihazının girişlerine bağlanır. Klasik kumanda devrelerinde motor vb. alıcılara yol vermek için kullanılan kontaktörler ise PLC çıkışlarına bağlanır. Klasik kumanda sistemlerinde rölelerle birlikte karmaşık devre bağlantıları ile gerçekleştirilen işlemler PLC ile program yazılarak kolayca gerçekleştirilebilir.

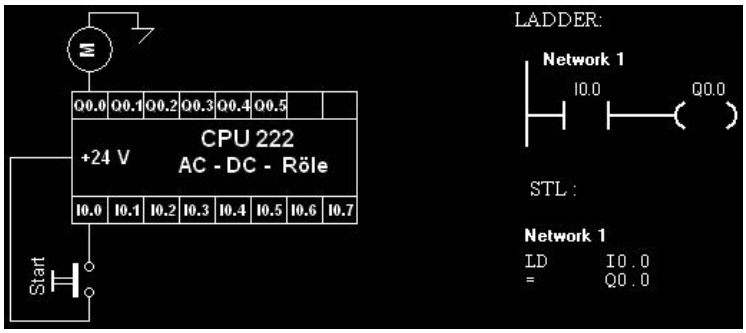
Bu bölümde klasik kumanda devreleri ile Ladder dili kullanılan PLC programları arasındaki ilişki anlatılacaktır.

Start Butonu ile Bir Motorun Kesik Çalıştırılması

Şekildeki klasik kumanda devresinde kullanılan start butonu ani temaslıdır. Butona basıldığında M kontaktörü enerjilenir ve yol verdiği motor çalışır. Butondan elimizi çektiğimizde M kontaktörünün enerjisi kesilir ve yol verdiği motorun çalışması durur.



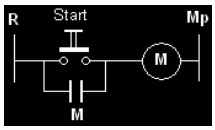
Şekildeki klasik kumanda devresini PLC ile gerçekleştirmek için start butonunun PLC girişlerinden herhangi birine, M kontaktörünün ise PLC çıkışlarından herhangi birine bağlanması gerekir. Burada kullanılan PLC CPU 222 AC –DC – Röle modeli olup çıkışlarından 220 V altında 2A'e kadar akım çekilebilmektedir. Bu işlemden sonra yapılması gereken PLC programı ile Start butonunun bağlı olduğu giriş adresi ile M kontaktörünün bağlı olduğu çıkış adresini istenilen şartlara göre ilişkilendirmektir.

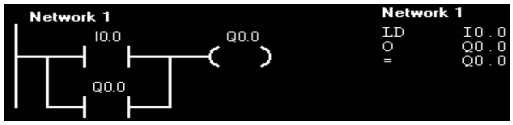


Şekilde verilen PLC programında I0.0 girişine bağlı start butonuna basıldığında I0.0 giriş adres biti lojik 1 değerini alır. I0.0 giriş biti ile adreslenen normalde açık kontak kapanarak Q0.0 çıkış adres bitinin lojik 1 değerini almasını sağlar. Q0.0 çıkışı aktif olur ve M kontaktörü enerjilenerek yol verdiği motor çalışmaya başlar. Start butonundan elimizi çektiğimizde I0.0 giriş adres biti lojik 0 değerini alır ve bu bitle adreslenen kontak açılar. Q0.0 çıkış adres biti lojik 0 değerini alır. Bu durumda M kontaktörünün enerjisi kesilerek yol verdiği motorun çalışması durur.

Start butonu ile bir motorun sürekli çalıştırılması

Şekildeki klasik kumanda devresinde start butonuna basıldığında M kontaktörü enerjilenerek kontaklarını konum değiştirir. Start butonundan elimizi çektiğimizde M kontaktörü, kapanmış olan kontağı üzerinden enerjili kalmaya devam eder ve yol verdiği motor sürekli çalışır. Bu olaya mühürleme adı verilir.

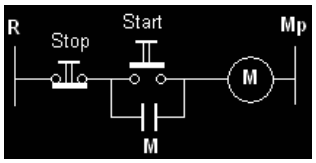




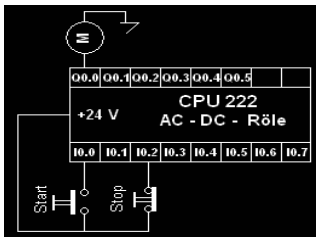
Şekildeki klasik kumanda devresini PLC ile gerçekleştirmek için gerekli bağlantı diğer şekil ile aynıdır. Burada mühürleme işlemi program tarafından gerçekleştirilir. Şekilde görülen PLC programında I0.0 girişine bağlı start butonuna basıldığında bu giriş ile adreslenen kontak kapanarak Q0.0 çıkışını enerjilendirir. Q0.0 çıkış biti lojik 1 değerini alarak bu bitle adreslenen kontak kapanır ve mühürleme işlemini gerçekleştirir. Butondan elimizi çeksek bile çıkış enerjili kalmaya devam eder. PLC programlamada mühürleme işlemi, daha sonraki bölümlerde anlatılacak olan SET fonksiyonu ile de gerçekleştirilebilir. Bu devrede mühürleme işlemi ile çıkış sürekli enerjili kalır ve çıkışa bağlı alıcı durdurulamaz. Bu nedenle sistemi durdurmak için PLC girişine stop butonu bağlayarak programla ilişkilendirmemiz gerekir.

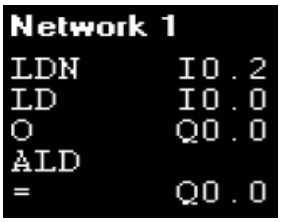
Start butonu ile sürekli çalıştırılan motorun stop butonu ile durdurulması

Şekildeki klasik kumanda devresinde start butonuna basıldığında M kontaktörü enerjilenerek kendisini mühürler. Start butonundan elimizi çeksek dahi M kontaktörü enerjili kalmaya devam eder. Stop butonuna basıldığında M kontaktörünün enerjisi kesilerek sistemin çalışması sona erer.

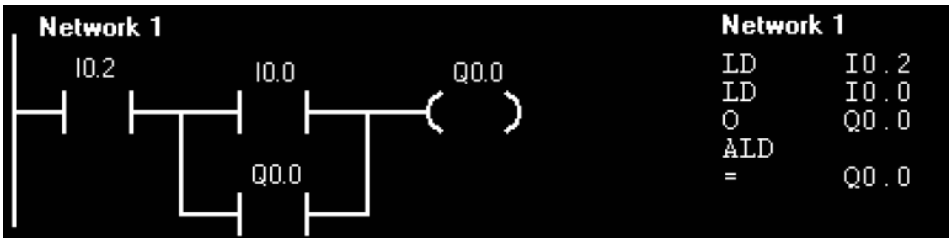


Şekildeki devreyi PLC ile gerçekleştirmek için Şekilde PLC'nin I0.2 girişine bir stop butonu bağlanmıştır. Sistemi durdurmak için PLC programında aşağıdaki ilk Şekildeki gibi normalde kapalı bir kontak kullanırsak önemli bir hata yapmış oluruz. Çünkü I0.2 girişine bağlı stop butonu normalde kapalı pozisyonda olduğundan bağlı olduğu girişi sürekli enerji altında tutar. I0.2 giriş biti sürekli lojik 1 konumda olacağından ladder programındaki I0.2 giriş biti ile adreslenmiş normalde kapalı kontak konum değiştirerek sürekli açık durumda kalacaktır. Bu durumda start butonu ile sistem çalıştırılamaz.





Bu sakıncayı önlemek için sistemi durdurmak amacıyla kullanılan I0.2 kontağının Şekildeki gibi normalde açık olması gerekir. Eğer sistemi durdurmak için kullanılan PLC girişinin sürekli enerji altında kalmasını istemiyorsak durdurma işlemini start butonu kullanarak yapabiliriz.



2.8.2. Çıkış Atama (=) Komutu

PLC programlarında mantıksal sinyal akışının nihai sonucunu bir adres bitine aktarmak için çıkış komutu kullanılır. Sinyalin aktarıldığı bit lojik 1 değerini alır. Eğer sinyal akışı bir çıkış bitini lojik 1 yaparsa bu çıkışa bağlı alıcı enerjilenir. Sinyal akışı kesildiğinde çıkış ataması yapılan bit lojik 0 değerine döner.



ÇIKIŞ KOMUTU OPERANTLARI: **I, Q, M, SM, S, T, C, V, L hafıza alan bitleri**

Ladder yöntemiyle yazılan PLC programlarında bir çıkış doğrudan enerji kaynağına bağlanamaz. Diğer bir ifade ile program bir çıkış atama komutu ile başlayamaz. Çıkıştan önce en az bir tane normalde açık ya da normalde kapalı kontak bulunmalıdır.



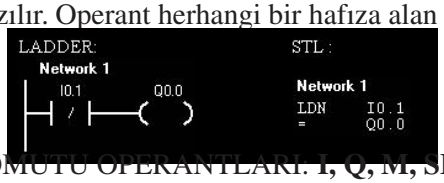
2.8.3. LD ve LDN Kontak Yükleme Komutları

LD Komutu: Normalde açık bir kontak ile hat (komut bloğu) başlatılmasını sağlar. Kontağın üst kısmına operant yazılır. Operant herhangi bir hafıza alan biti olabilir. Yükleme komutu ile başlayıp bir sonraki yükleme komutuna kadar olan kesim, mantıksal olarak birbirine bağlı komut bloğudur.

OPERANTLAR: **I, Q, M, SM, S, T, C, V, L hafıza alan bitleri**



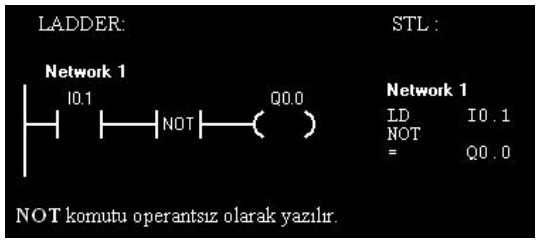
LDN Komutu: Normalde kapalı bir kontak ile hat başlatılmasını sağlar. Kontağın üst kısmına operant yazılır. Operant herhangi bir hafıza alan biti olabilir.



LD VE **LDN** KOMUTU OPERANTLARI: **I, Q, M, SM, S, T, C, V, L hafıza alan bitleri**

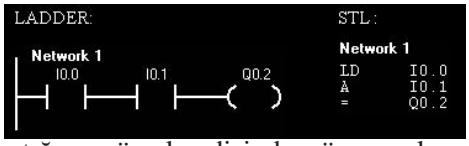
2.8.4. NOT Değilleme

NOT komutu ladder diygramlarında NOT kontağı ile gösterilir. NOT kontağı enerji akışını tersine çevirir. Girişinde enerji varsa çıkışında enerji olmaz, eğer girişinde enerji yoksa çıkışına enerji sağlar. Yapılan işlem lojik değil işlemidir. NOT kontağı enerji hattına doğrudan bağlanamaz. Aşağıda örnekte NOT kontağının kullanım şekli ve STL dilindeki kullanımı görülmektedir.

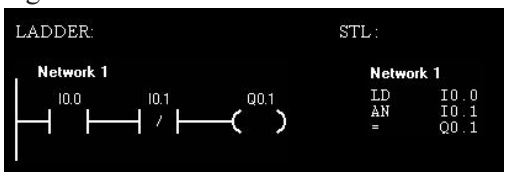


2.8.5. A (VE) , AN (VE DEĞİL) Komutları ve Uygulaması

A Komutu ladder mantığına göre kendisinden önce gelen devre kesimine normalde açık bir kontağı seri bağlar.



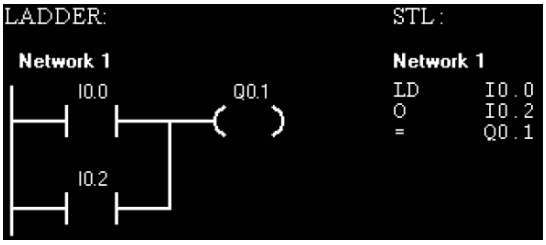
AN Komutu ladder mantığına göre kendisinden önce gelen devre kesimine normalde kapalı bir kontağı seri bağlar.



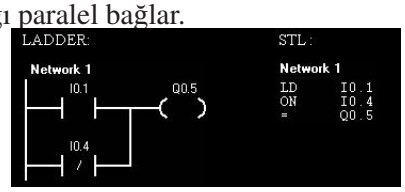
A ve **AN** KOMUTLARI OPERANTLARI: **I, Q, M, SM, S, T, C, V, L hafıza alan bitleri**

2.8.6. O (VEYA), ON (VEYA DEĞİL) Komutları ve Uygulaması

O Komutu ladder mantığına göre kendisinden önce gelen devre kesimine normalde açık bir kontağı paralel bağlar.

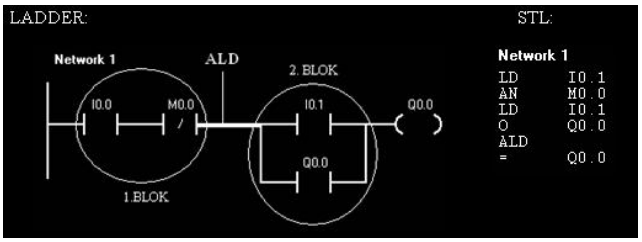


ON Komutu ladder mantığına göre kendisinden önce gelen devre kesimine normalde kapalı bir kontağı paralel bağlar.



2.8.7. Birden Fazla Komut Bloğu Arasında (ALD) VE İşlemi ve Uygulaması

Ladder mantığına göre birden fazla kontağın kendi arasında seri veya paralel bağlanması ile oluşan gruba blok adı verilir. STL mantığına göre blok bir kontak yükleme (LD veya LDN) komutuyla başlayan ve bir sonraki kontak yükleme komutuna kadar olan kesimdir. Birden fazla komut bloğunu birbirine seri bağlamak için (VE işlemi) ALD komutu kullanılır.

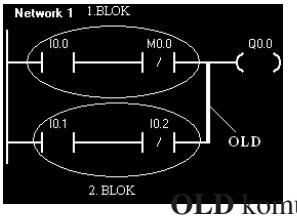


ALD komutu operantsız olarak yazılır.

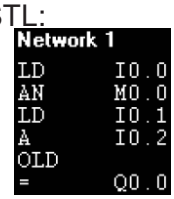
2.8.8. Birden Fazla Komut Bloğu Arasında (OLD) VEYA İşlemi ve Uygulaması

Birden fazla komut bloğunu birbirine paralel bağlamak için OLD komutu kullanılır.

LADDER:

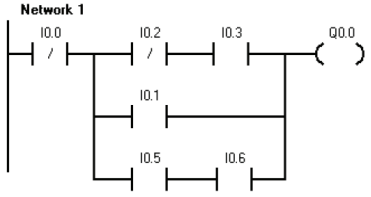


STL:



OLD komutu operantsız olarak yazılır.

UYGULAMA 1: Şekilde ladder diyagramı verilen PLC programını STL dili ile yazınız?



Çözüm:

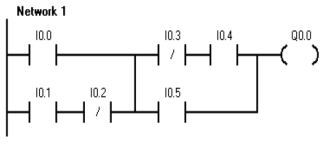
```
Network 1
LDN   I0.0
LDN   I0.2
A     I0.3
O     I0.1
LD    I0.5
A     I0.6
OLD
ALD
=     Q0.0
```

UYGULAMA2: Verilen STL dili programının Ladder diyagramını çiziniz?

STL:

```
Network 1
LD    I0.0
LD    I0.1
AN    I0.2
OLD
LDN   I0.3
A     I0.4
O     I0.5
ALD
=     Q0.0
```

Çözüm:

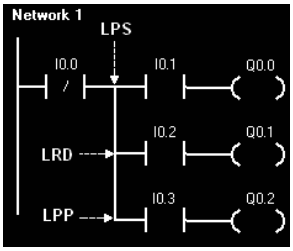


2.8.9. LPS, LPP, LRD (Lojik yığın) Komutları

LPS Komutu: Ara kol bağlantısı oluşturmak için kullanılır.
LRD Komutu: Bir çıkışa arakol oluşturmak için kullanılır.
LPP Komutu: En son çıkış kolu oluşturmak için kullanılır.

ÖRNEK:

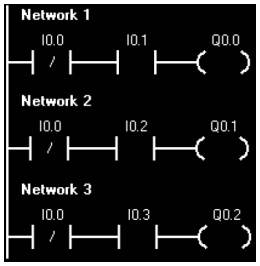
LADDER:



STL:

```
Network 1
LDN     I0.0
LPS
A       I0.1
=       Q0.0
LRD
A       I0.2
=       Q0.1
LPP
A       I0.3
=       Q0.2
```

LADDER:



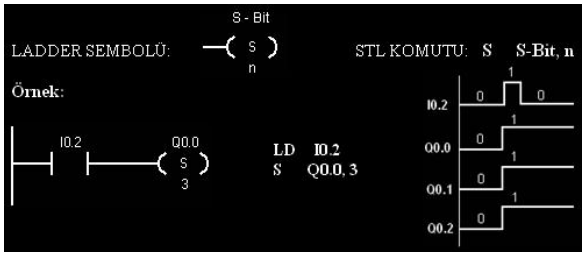
STL:

```
Network 1
LDN     I0.0
A       I0.1
=       Q0.0
Network 2
LDN     I0.0
A       I0.2
=       Q0.1
Network 3
LDN     I0.0
A       I0.3
=       Q0.2
```

Kolların farklı network'lere yerleştirilmesi durumunda lojik yığın komutlarına ihtiyaç kalmaz.

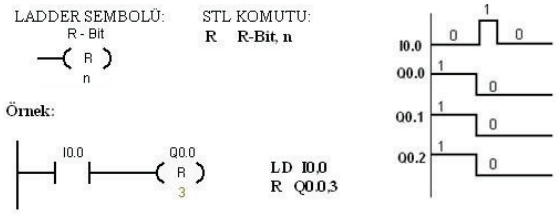
2.8.10. SET, RESET Komutları ve Uygulaması

SET KOMUTU: Bir hafıza alanının istenilen bitinden (S-Bit) itibaren n adet biti set (lojik 1) yapmak için kullanılır. Set yapılan çıkışlar hafızaya alınır ve reset yapılana kadar aktif durumda kalır.



Verilen ladder devresinde I0.2 girişi aktif (lojik1) olduğunda kontak kapanarak SET fonksiyonunu çalıştırır. SET fonksiyonu Q0.0 dan itibaren 3 biti, bir başka ifade ile Q0.0, Q0.1 ve Q0.2 çıkış bitlerini aktif yapar. PLC'nin bu çıkışlarına bağlı alıcılar çalışır. I0.2 girişi lojik 0 olduğunda kontak açılır. Set yapılan çıkışlar hafızaya alındığından aktif durumda kalmaya devam eder. PLC programlamada klasik mühürleme tekniği yerine kullanılır.

RESET KOMUTU: Bir hafıza alanının istenilen bitinden (S-Bit) itibaren n adet biti reset (lojik 0) yapmak için kullanılır.



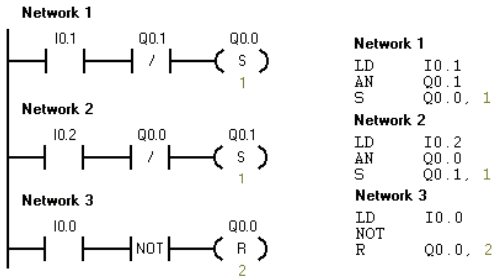
Verilen ladder devresinde I0.0 girişi aktif (lojik 1) yapıldığında kontak kapanarak Reset fonksiyonunu çalıştırır. Reset fonksiyonu Q0.0 dan itibaren 3 biti, bir başka ifade ile Q0.0, Q0.1 ve Q0.2 çıkış bitlerini pasif (lojik 0) yapar. PLC'nin bu çıkışlarına bağlı alıcıların çalışması durur. Reset komutunun etkisini görmek için bitlerin set yapılmış olması gerekir.

SET VE RESET KOMUTLARI OPERANTLARI:

S-Bit, R-Bit : **I,Q,M, SM,S,T,C,V,L hafıza alan bitleri**
n : **IB, QB, VB, MB, SMB, SB, LB, AC, * VD, * LD, * AC, Sabit (0 ... 255)**

UYGULAMA: Üç fazlı bir asenkron motor ileri geri çalıştırılacaktır. I0.1 girişine bağlı start butonuna basıldığında motor ileri yönde, I0.2 girişine bağlı start butonuna basıldığında motor geri yönde çalışacaktır. Motor herhangi bir yönde çalışırken diğer yönde çalıştırılamayacaktır. I0.0 girişine bağlı stop butonuna basıldığında motor duracaktır.

Çözüm:

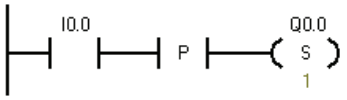


2.8.11. Giriş Sinyalinin Çıkan Kenarına Göre Çıkış Alma

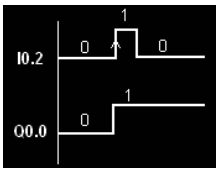
Yükselen kenar (Pozitif kenar) algılama kontağı girişine uygulanan sinyalin 0'dan 1 konumuna her geçişinde sadece bir tarama için enerji akışına izin verir. Pozitif kenar algılama kontağının STL eşdeğeri EU komutudur.

Ladder Sembolü: —| P |— STL Komutu: **EU**

Örnek:




```
LD I0.0
EU
S Q0.0,1
```



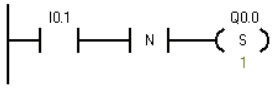
Verilen Ladder devresinde I0.0 girişi ani temaslı bir butonla aktif yapıldığında kontak kapanarak enerji akışı sağlar. Ancak grafikten anlaşılacağı gibi pozitif kenar algılama kontağı I0.0 kontağından gelen sinyalin sadece yükselen kenarında enerji akışına izin verererek Q0.0 çıkış bitinin set yapılmasını sağlar.

2.8.12. Giriş Sinyalinin İnen Kenarına Göre Çıkış Alma ve Uygulaması

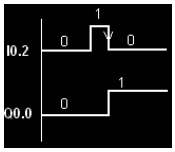
Düşen kenar (Negatif kenar) algılama kontağı girişine uygulanan sinyalin 1'den 0 konumuna her geçişinde sadece bir tarama için enerji akışına izin verir. Negatif kenar algılama kontağının STL eşdeğeri ED komutudur.

Ladder Sembolü: —| N |— STL Komutu: **ED**

Örnek:



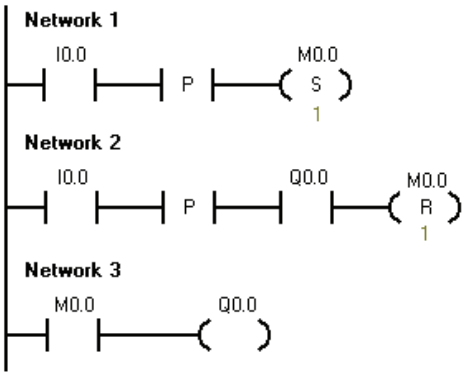

```
LD I0.0
ED
S Q0.0
```



Şekildeki ladder devresinde I0.1 girişi ani temaslı bir butonla aktif yapıldığında kontak kapanarak enerji akışı sağlar. Ancak grafikten anlaşılacağı gibi negatif kenar algılama kontağı I0.1 kontağından gelen sinyalin sadece düşen kenarında enerji akışına izin vereceğinden çıkış enerjilenmez. Butondan elimizi çektiğimiz anda negatif kenar algılama kontağı girişine uygulanan sinyalin düşen kenarında enerji akışına izin vererek Q0.0 çıkış bitinin set yapılmasını sağlar. Pozitif ve Negatif kenar algılama komutları operant kullanmaz.

UYGULAMA: Q0.0 çıkışı ile kontrol edilen bir motor I0.0 girişine bağlı ani temaslı bir start butonu ile hem çalıştırılıp hem de durdurulacaktır.

Çözüm:




```
Network 1
LD      I0.0
EU
S       M0.0, 1

Network 2
LD      I0.0
EU
A       Q0.0
R       M0.0, 1

Network 3
LD      M0.0
=       Q0.0
```

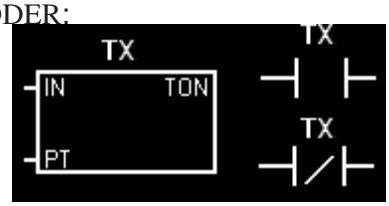

2.8.13. Zamanlayıcı (Timer) Komutu

PLC programlarında bir giriş sinyaline karşılık zamana bağımlı çıkış sinyali üretmek için zamanlayıcılar kullanılır. S7 – 200 PLC'lerde çalışma şekli bakımından 3 tip zamanlayıcı bulunmaktadır.

Çekmede Geçikmeli Zamanlayıcı (TON)

IN girişine enerji geldikten sonra PT girişi ile belirlenen zaman sonunda, zamanlayıcı biti ile adreslenen kontakları konum değiştiren zamanlayıcılardır. Klasik kumanda devrelerindeki düz zaman rölesi gibi çalışır.

LADDER:



STL:
TON TX ,PT

•**IN:** Zamanlayıcı girişidir. IN girişine gelen sinyal 1 olduğunda zamanlayıcı zamanı saymaya başlar. Bu giriş doğrudan enerji hattına bağlanamaz. Zamanlayıcıyı başlatmak için mutlaka bir kontak kullanılmalıdır.

• **PT(Preset Time):** Zamanlayıcı ayar değeridir ve kullanıcı tarafından belirlenir. PT değeri desimal olarak 0….32767 aralığında herhangi bir değer olabilir. PT değerini belirlemek için bir hafıza alan adresi kullanılabilir.

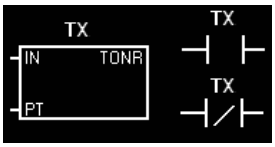
• **TX:** Zamanlayıcı adresini belirtir. Zamanlayıcı tarafından sayılan süre anlık olarak bu adrese yazılır. Aynı adresi kullanan bir de zamanlayıcı biti bulunmaktadır. IN girişinin enerjilenmesi ile birlikte zamanlayıcı zaman sayma işlemini başlatır. TX adresine kaydedilen zamanlayıcı anlık değeri ile PT ayar değeri birbirine eşit olduğu anda zamanlayıcı biti (TX) ile adreslenen kontaklar konum değiştirir. Zaman sayma işlemi 32767'ye kadar devam eder. Eğer zamanlayıcının IN girişine gelen sinyal 0 olursa sayma değeri sıfırlanır.

Çekmede Gecikmeli Kalıcı Tip (toplamalı tip) Zamanlayıcı (TONR)

TONR tipi zamanlayıcı TON tipi zamanlayıcıdan farklı olarak enerjili kaldığı sürelerin toplamını sayar. Zamanlayıcıda IN girişi enerjilendiğinde zaman sayma işlemi başlar. Eğer PT değerine ulaşılmadan IN girişine gelen enerji kesilirse zaman rölesi anlık değeri zaman rölesi adresinde saklanır. Böylece enerji tekrar uygulandığında zamanlayıcı sayma işlemine kaldığı yerden devam eder. Herhangi bir anda TX adresindeki zamanlayıcı anlık değeri ile PT değeri eşit olduğunda zamanlayıcı biti ile adreslenen kontaklar konum değiştirir.

TONR tipi zamanlayıcının değerini sıfırlamak için zamanlayıcı bitinin RESET komutuyla sıfırlanması gerekir.

LADDER SEMBOLÜ:



STL KOMUTU:

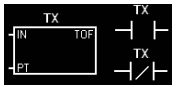
TONR TX, PT

Bırakmada Geçikmeli Zamanlayıcı (TOF)

TOF tipi zamanlayıcı IN girişi enerjilendiğinde zamanlayıcı biti ile adreslenen kontakları hemen konum değiştirir. IN girişine uygulanan sinyal kesildiği anda zamanlayıcı sayma işlemini başlatır. TX adresine yazılan zamanlayıcı anlık değeri ile PT ayar değeri birbirine eşit olduğunda kontaklar normal konumlarına dönerler. Klasik kumanda devrelerinde kullanılan ters zaman rölesi gibi çalışır.

TOF tipi bırakmada gecikmeli zamanlayıcı her marka ve model PLC'de bulunmayabilir. TOF tipi zmanlayıcı komutu bulunmayan bir PLC'de program hazırlarken ihtiyaç duyulması halinde problem TON tipi zamanlayıcılarla çözülür.

LADDER SEMBOLÜ:



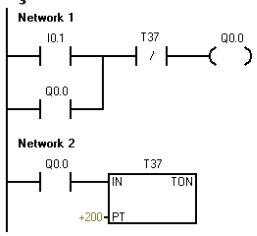
STL KOMUTU:
TOF TX,PT

ZAMANLAYICI KOMUTLARI OPERANTLARI:
IN: **I,Q,M, SM,S,T,C,V,L** hafıza alan bitleri ile adreslenen bir kontaktan sonra bağlanır.
TX: **T0** … **T255** arasındaki zamanlayıcı adresleri
PT: **IW, QW, VW, MW, SMW, SW, LW, AIW, AC, T, C, * VD, * LD, * AC, Sabit (0 ... 32767)**

Zamanlayıcı Uygulamaları

UYGULAMA 1: I0.1 girişine bağlı start butonuna basıldığında Q0.0 çıkışına bağlı motor çalışarak 20 sn sonra kendiliğinden duracaktır. Gerekli PLC programını yazınız?

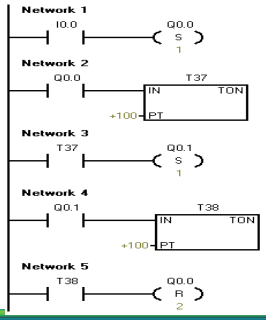
Çözüm:




```
Network 1
LD      I0.1
O       Q0.0
AN      T39
=       Q0.0
Network 2
LD      Q0.0
TON     T37, +200
```

UYGULAMA 2: I0.1 girişine bağlı start butonuna basıldığında Q0.0 çıkışına bağlı 1.motor çalışmaya başlayacak, 10 sn sonra Q0.1 çıkışına bağlı 2.motor devreye girecektir. İki motor birlikte 20 sn çalıştıktan sonra her iki motor devreden çıkacaktır. Gerekli PLC programını yazınız?

Çözüm:




```
Network 1
LD      I0.0
S       Q0.0, 1
Network 2
LD      Q0.0
TON     T37, +100
Network 3
LD      T37
S       Q0.1, 1
Network 4
LD      Q0.1
TON     T38, +100
Network 5
LD      T38
R       Q0.0, 2
```

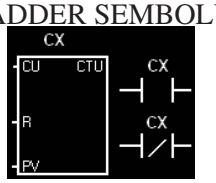
Sayıcı (Counter) Komutları

Endüstride, üretilen ürün sayılarının belirlenmesi ya da bir işlemin tekrar sayısının tespitinde sayıcılar kullanılır. Sayıcılar sayma girişlerine gelen sinyalin yükselen kenarlarını sayarlar.
S7 – 200 PLC'lerde çalışma şekli bakımından üç tip sayıcı bulunmaktadır.

Yukarı Sayıcı (CTU)

CTU sayıcıda CU sayma girişine uygulanan sinyalin her yükselen kenarlarında sayıcı anlık değeri 0 dan itibaren bir artar. Sayıcı anlık değeri CX ile belirtilen sayıcı adresinde saklanır. Herhangi bir anda sayıcı anlık değeri, PV sayma değerine eşit ya da büyük olduğunda sayıcı biti (CX) ile adreslenen kontaklar konum değiştirir.

LADDER SEMBOLÜ:



STL KOMUTU:

CTU CX,PV

Şekilde görülen CTU tipi sayıcıda;

• **CU (Count UP):** Yukarı sayma girişidir. CU girişine uygulanan bir yükselen kenar sinyali sayıcı anlık değerini bir arttırır. Bu giriş doğrudan enerji hattına bağlanamaz. Sayma sinyallerini üreten bir kontak ile kullanılır.

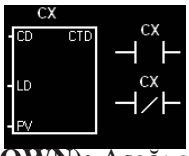
• **R (Reset):** Sayıcı resetleme(sıfırlama) girişidir. R girişine 1 uygulandığında CX adresinde bulunan sayıcı anlık değeri ve sayıcı biti sıfırlanır. Konum değiştirmiş olan sayıcı kontakları normal konumlarına döner. CU girişine sinyal uygulanmaya devam ediliyorsa sayıcı sıfırdan itibaren tekrar saymaya başlar.

• **PV (Preset Value):** Sayıcı ayar değeridir ve kullanıcı tarafından belirlenir. PV değeri desimal olarak 0….32767 aralığında herhangi bir değer olabilir. PV değerini belirlemek için bir hafıza alan adresi kullanılabilir.

Aşağı sayıcı (CTD)

CTD sayıcıda CD sayma girişine uygulanan sinyalin her yükselen kenarlarında sayıcı anlık değeri LD girişi ile yüklenen sayma değerinden itibaren bir azalır. Sayıcı anlık değeri CX ile belirtilen sayıcı adresinde saklanır. Herhangi bir anda sayıcı anlık değeri 0 olduğunda sayıcı biti (CX) ile adreslenen kontaklar konum değiştirir.

LADDER SEMBOLÜ:



STL KOMUTU:

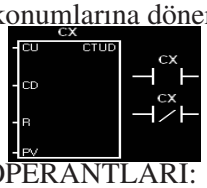
CTD CX,PV

- **CTD (Count DOWN):** Aşağı sayma girişidir. CD girişine uygulanan bir yükselen kenar sinyali sayıcı anlık değerini bir azaltır. Bu giriş doğrudan enerji hattına bağlanamaz. Sayma sinyallerini üreten bir kontak ile kullanılır.
- **LD (Reset):** Yükleme girişidir. Bu girişe uygulanacak bir yükselen kenar sinyali, PV ile belirtilen sayıcı ayar değerini CX ile belirtilen sayıcı adresine yükler.
- **PV (Preset Value):** Sayıcı ayar değeridir ve kullanıcı tarafından belirlenir. PV değeri desimal olarak 0….32767 aralığında herhangi bir değer olabilir. PV değerini belirlemek için bir hafıza alan adresi kullanılabilir.

Aşağı yukarı Sayıcı (CTUD)

CTUD sayıcıda CU (yukarı sayma) girişine uygulanan sinyalin her yükselen kenarında sayıcı anlık değeri bir artarken, CD (aşağı sayma) girişine uygulanan sinyalin her yükselen kenarında sayıcı anlık değeri bir azalır. Sayıcı anlık değeri CX ile belirtilen sayıcı adresinde saklanır. Herhangi bir anda sayıcı anlık değeri, PV sayma değerine eşit ya da büyük olduğunda sayıcı biti (CX) ile adreslenen kontaklar konum değiştirir. R girişi sayıcı resetleme(sıfırlama) girişidir. R girişine lojik 1 uygulandığında CX adresinde bulunan sayıcı anlık değeri ve sayıcı biti sıfırlanır. Konum değiştirmiş olan sayıcı kontakları normal konumlarına döner.

LADDER SEMBOLÜ



STL KOMUTU:
CTD CX,PV

SAYICI KOMUTLARI OPERANTLARI:

CU, CD, R : **I, Q, M, SM, S, T, C, V, L** hafıza alan bitleri ile adreslenen bir kontaktan sonra bağlanır.

CX: **C0** … **C255** arasındaki sayıcı adresleri

PT: **IW, QW, VW, MW, SMW, SW, LW, AIW, AC, T, C, * VD, * LD, * AC, Sabit (-32768 ... 32767)**

2.8.19. Karşılaştırma Komutları ve Uygulaması

Karşılaştırma komutları iki veriyi büyüklük, küçüklük ve eşitlik ölçütlerine göre kıyaslayarak gerekli kumanda işaretlerinin üretilmesini sağlarlar. Karşılaştırma komutları ladder diyagramlarında bir karşılaştırma kontağı ile gösterilir.

Bayt karşılaştırma komutları

Bayt uzunluğundaki (0 – 255) iki operant içeriği karşılaştırılır. Operantları bayt uzunluğundaki herhangi bir hafıza alan adresi ve sabit değer olabilir. Karşılaştırma kontaklarının devreye bağlantı mantığı normal kontaklar gibidir. Bu yüzden STL dili programlarında karşılaştırma kontakları LD (Başlangıç), A (Seri bağlı), ve O (Paralel)bağlı konumda olabilir.

LADDER SEMBOL:	STL :	
n1 ==B n2	LDB= n1,n2 AB= n1,n2 OB= n1,n2	n1 değeri n2 değerine eşit olduğunda karşılaştırma kontağı kapanarak enerji akışına izin verir.
n1 <=B n2	LDB<= n1,n2 AB<= n1,n2 OB<= n1,n2	n1 değeri n2 değerine eşit ya da n2 değerinden küçük olduğunda karşılaştırma kontağı kapanarak enerji akışına izin verir.
n1 >=B n2	LDB>= n1,n2 AB>= n1,n2 OB>= n1,n2	n1 değeri n2 değerine eşit ya da n2 değerinden küçük olduğunda karşılaştırma kontağı kapanarak enerji akışına izin verir.
n1 <>B n2	LDB< > n1,n2 AB< > n1,n2 OB< > n1,n2	n1 değeri n2 değerine eşit değilse karşılaştırma kontağı kapanarak enerji akışına izin verir.
n1 >B n2	LDB > n1,n2 AB > n1,n2 OB > n1,n2	n1 değeri n2 değerinden büyük olduğunda karşılaştırma kontağı kapanarak enerji akışına izin verir.
n1 <B n2	LDB < n1,n2 AB < n1,n2 OB < n1,n2	n1 değeri n2 değerinden küçük olduğunda karşılaştırma kontağı kapanarak enerji akışına izin verir.

Tamsayı karşılaştırma komutları

Bir word uzunluğundaki (-32768 … +32767) tamsayıların karşılaştırılmasında kullanılırlar. Operantları word uzunluğundaki herhangi bir hafıza alan adresi ve sabit değer olabilir. Tamsayı karşılaştırma komutlarının Ladder ve STL gösterim biçimi alan belirteci dışında aynıdır.

LADDER	STL :	LADDER :	STL :
n1 ==I n2	LDW= n1,n2 AW= n1,n2 OW= n1,n2	n1 <>I n2	LDW< > n1,n2 AW< > n1,n2 OW< > n1,n2
n1 <=I n2	LDW<= n1,n2 AW<= n1,n2 OW<= n1,n2	n1 >I n2	LDW > n1,n2 AW > n1,n2 OW > n1,n2
n1 >=I n2	LDW>= n1,n2 AW>= n1,n2 OW>= n1,n2	n1 <I n2	LDW < n1,n2 AW < n1,n2 OW < n1,n2

Double word uzunluğundaki verilerin karşılaştırılması için alan belirteci D olan Double word karşılaştırma komutları, Double word uzunluğundaki alanda bulunan reel sayıları karşılaştırmak için ise alan belirteci R olan karşılaştırma komutu kullanılır.

KARŞILAŞTIRMA KOMUTLARINDA N1 ve N2 İÇİN OPERANTLAR:

BAYT KARŞILAŞTIRMA:
IB, QB, VB, MB, SMB, SB, LB, AC, * VD, * LD, * AC, Sabit

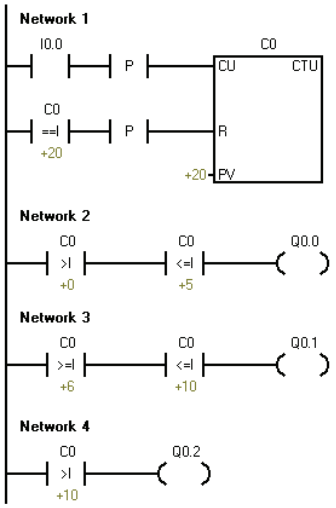
WORD TAMSAYI KARŞILAŞTIRMA:
IW, QW, VW, MW, SMW, SW, LW, AIW, AC, T, C, * VD, * LD, * AC, Sabit

DOUBLE WORD TAMSAYI KARŞILAŞTIRMA:
ID, QD, VD, MD, SMD, SD, LD, AC, HD, * VD, * LD, * AC, Sabit

REEL SAYI KARŞILAŞTIRMA:
ID, QD, VD, MD, SMD, SD, LD, AC, * VD, * LD, * AC, Sabit

Uygulama: I0.0 girişine bağlı butona her basılışta yukarı sayıcı değeri bir artacaktır. Sayıcı içeriği 0 ile 5 arasında (5 dahil) ise Q0.0 çıkışı, 6 ile 10 arasında ise Q0.1 çıkışı, eğer sayıcı içeriği 10'dan büyükse Q0.2 çıkışı aktif olacaktır. Sayıcı içeriği 20 olduğunda sayıcı sıfırlanacaktır.

Çözüm:




```
Network 1
LD      I0.0
EU
LDW=    C0, +20
EU
CTU     C0, +20

Network 2
LDW>    C0, +0
AW<=    C0, +5
=       Q0.0

Network 3
LDW>=   C0, +6
AW<=    C0, +10
=       Q0.1

Network 4
LDW>    C0, +10
=       Q0.2
```


2.8.20.2. Taşıma (MOVE) Komutları

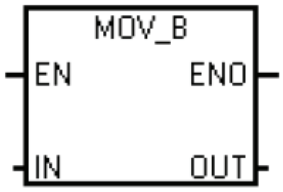
Bayt, Word ve Double Word uzunluğundaki sabit verilerin aynı uzunluktaki bir hafıza alanına ya da bir hafıza alanında bulunan Bayt, Word ve Double Word uzunluğundaki verinin aynı uzunluktaki başka bir hafıza alanına taşımak için kullanılan komutlardır.

Move komutları kaynak ve hedef olmak üzere iki operant kullanır. Taşınacak verinin bulunduğu alan kaynak, taşınacak verinin yazılacağı alan ise hedef operanttır. Move komutlarının IN girişlerine kaynak, OUT çıkışlarına ise hedef operant yazılır. Taşıma işleminin gerçekleşmesi için komutların EN girişlerinin enerjilenmesi gerekir. EN girişleri kesinlikle doğrudan enerji kaynağına bağlanamaz. Bu işlem bir kontak ile gerçekleştirilmelidir.

Bayt Taşıma (MOV_B) Komutu

EN girişi enerjilendiğinde, IN girişi ile belirtilen kaynak operanttaki bayt uzunluğunda veriyi OUT çıkışı ile belirtilen yine bayt uzunluğundaki hedef operant alanına taşır. Bu taşımada kaynak operant içeriğinde herhangi bir değişiklik olmaz. Taşıma komutunun icrası sırasında bir hata oluşursa ENO çıkışı lojik 0 değerini alır.

LADDER:



STL: **MOVB IN, OUT**

OPERANTLAR:

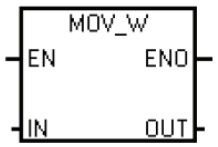
IN: **IB, QB, VB, MB, SMB, SB, LB, AC, * VD, * LD, * AC Sabit**

OUT: **IB, QB, VB, MB, SMB, SB, LB, AC,* VD, * LD, * AC**

Word Taşıma (MOV_W) Komutu

EN girişi enerjilendiğinde, IN girişi ile belirtilen kaynak operanttaki Word uzunluğunda veriyi OUT çıkışı ile belirtilen yine word uzunluğundaki hedef operant alanına taşır. Bu taşımada kaynak operant içeriğinde herhangi bir değişiklik olmaz. Taşıma komutunun icrası sırasında bir hata oluşursa ENO çıkışı lojik 0 değerini alır.

LADDER:



STL: **MOVB IN, OUT**

OPERANTLAR:

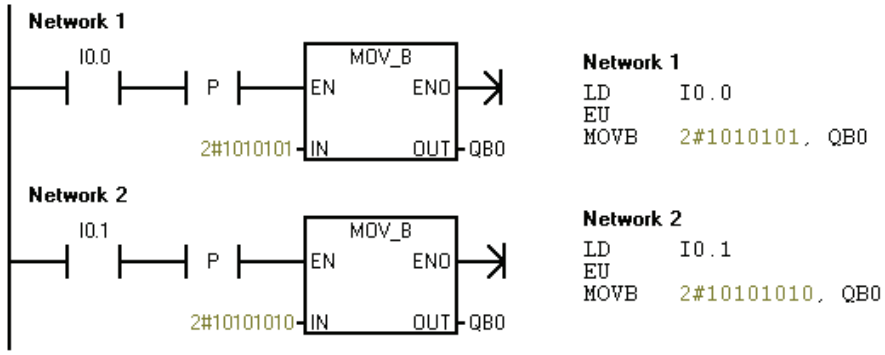
IN: **IW, QW, VW, MW, SMW, SW, LW, AIW, AC, T, C, Sabit**

OUT: **IW, QW, VW, MW, SMW, SW, LW, AQW, AC, T, C * VD, * LD, * AC,**

Double word uzunluğundaki verileri taşımak için MOV_D, double word uzunluğundaki alanda bulunan reel sayıları taşımak için ise MOV_R komutu kullanılır. Bu komutların kullanım şekli ve operantları için S7 – 200 kullanım kılavuzuna bakınız.

Uygulama: I0.0 girişine bağlı start butonuna basıldığında çift numaralı çıkışlar (Q0.0, Q0.2, Q0.4, Q0.6) I0.1 girişine bağlı start butonuna basıldığında ise tek numaralı çıkışlar (Q0.1, Q0.3, Q0.5, Q0.7) aktif olacaktır.

Çözüm:



2.8.21. Matematik İşlem, Fonksiyon Komutları ve Uygulaması

PLC cihazlarında toplama, çıkarma, çarpma, bölme, karekök alma gibi işlevleri yerine getirebilen matematik işlem ve fonksiyon komutları bulunmaktadır. Bu fonksiyonların kullanılmasıyla kumanda ve kontrol sistemlerinde uzunluk hesabı, alan hesabı, hacim hesabı devir sayısı ölçümü gibi işlemleri yapmak mümkün olmaktadır.

Matematik işlem komutlarının icra edilmesi sonucunda yapılan işlemin sonucuna bağlı olarak SMB1 özel hafıza baytı alanında bulunan bazı durum bitleri set (lojik 1) olur. Bu durum bitleri kullanılarak işlem sonuçlarına göre program akışı değiştirilebilir.

SM1.0: Matematiksel işlem yapan komutların icrası sonucunda elde edilen sonuç sıfır (0) ise bu bit set olur. Sıfır biti olarak anılır.

SM1.1: Matematiksel işlem yapan komutların icrası sonucunda taşma meydana gelmesi veya geçersiz nümerik değer saptanması durumunda bu bit set olur. Taşma biti olarak anılır.

SM1.2: Matematiksel işlem yapan komutların icrası sonucunda elde edilen sonuç negatif bir sayı ise bu bit set olur. İşaret biti olarak anılır.

SM1.3: Bölme komutlarının icrası sırasında bir sayıyı sıfır ile bölme girişimi varsa bu bit set olur. Sıfıra bölme hatası biti olarak anılır.

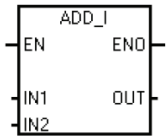
2.8.21.1. Matematik İşlem Komutları

Tamsayı Toplama (ADD_I) Komutu

Word uzunluğundaki iki tam sayının toplanması için kullanılır. Toplanacak sayılar IN1 ve IN2 girişlerine operant olarak yazılır. Operantlar sabit sayılar ve word uzunluğundaki hafıza alan adresleri olabilir. Toplam sonucu OUT çıkışı ile belirtilen word uzunluğundaki hedef operant alanına yazılır. (IN1+IN2 = OUT)

Toplama işleminin sonucuna bağlı olarak SM1.0, SM1.1 ve SM1.2 özel hafıza bitleri etkilenir. Toplama işlemi sırasında bir hata oluşursa ENO çıkışı lojik 0 değerini alır.

LADDER:



STL: **MOVW IN1, OUT**
+I IN2,OUT

OPERANTLAR:
IN1,IN2: **IW, QW, VW, MW, SMW, SW, LW, AIW, AC, T, C, *VD, *LD, *AC, Sabit**
OUT: **IW, QW, VW, MW, SMW, SW, LW, AIW, AC, T, C, * VD, *LD, *AC**

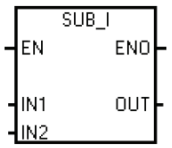
Double word uzunluğundaki tamsayıları toplamak için ADD_DI, Double word uzunluğundaki alanda bulunan reel sayıları toplamak ise ADD_R komutu kullanılır. Bu komutların kullanım şekli ve operantları için S7 – 200 kullanım kılavuzuna bakınız.

Tamsayı Çıkarma (SUB_I) Komutu

Word uzunluğundaki iki tam sayının çıkarılması için kullanılır. Çıkarılacak sayılar IN1 ve IN2 girişlerine operant olarak yazılır. Operantlar sabit sayılar ve word uzunluğundaki hafıza alan adresleri olabilir. Çıkarma işlemi sonucu OUT çıkışı ile belirtilen word uzunluğundaki hedef operant alanına yazılır. (IN1- IN2 = OUT)

Çıkarma işleminin sonucuna bağlı olarak SM1.0, SM1.1 ve SM1.2 özel hafıza bitleri etkilenir. Çıkarma işlemi sırasında bir hata oluşursa ENO çıkışı lojik 0 değerini alır.

LADDER:



STL: **MOVW IN1, OUT**
-I IN2,OUT

OPERANTLAR:
IN1,IN2: **IW, QW, VW, MW, SMW, SW, LW, AIW, AC, T, C, *VD, *LD, *AC, Sabit**
OUT: **IW, QW, VW, MW, SMW, SW, LW, AC, T, C, * VD, *LD, *AC**

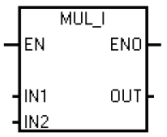
Double word uzunluğundaki tam sayıları çıkarmak için SUB_DI, double word uzunluğundaki alanda bulunan reel sayıları çıkarmak ise SUB_R komutu kullanılır.

Tamsayı Çarpma (MUL_I) Komutu

Word uzunluğundaki iki tam sayının çarpılması için kullanılır. Çarpılacak sayılar IN1 ve IN2 girişlerine operant olarak yazılır. Operantlar sabit sayılar ve word uzunluğundaki hafıza alan adresleri olabilir. Çarpım sonucu OUT çıkışı ile belirtilen word uzunluğundaki hedef operant alanına yazılır. (IN1* IN2 = OUT)

Çarpma işleminin sonucuna bağlı olarak SM1.0, SM1.1 ve SM1.2 özel hafıza bitleri etkilenir. Çarpma işlemi sırasında bir hata oluşursa ENO çıkışı lojik 0 değerini alır.

LADDER:



STL: **MOVW IN1, OUT**
***I IN2,OUT**

OPERANTLAR:
IN1,IN2: **IW, QW, VW, MW, SMW, SW, LW, AIW, AC, T, C, *VD, *LD, *AC, Sabit**
OUT: **IW, QW, VW, MW, SMW, SW, LW, AC, T, C, * VD, *LD, *AC**

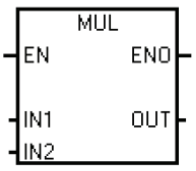
Double word uzunluğundaki tamsayıları çarpmak için MUL_DI, double word uzunluğundaki alanda bulunan reel sayıları çarpmak için ise MUL_R komutu kullanılır. Bu komutların kullanım şekli ve operantları için S7 – 200 kullanım kılavuzuna bakınız.

Double Tam Sayı Sonuçlu Tam Sayı Çarpma (MUL) Komutu

Bu komut IN1 ve IN2 girişlerinde bulunan word uzunluğundaki iki tam sayıyı çarpar ve sonucu OUT çıkışı ile belirtilen double word uzunluğundaki hafıza alanına yazar. (IN1* IN2 = OUT)

Çarpma işleminin sonucuna bağlı olarak SM1.0, SM1.1 ve SM1.2 özel hafıza bitleri etkilenir. Çıkarma işlemi sırasında bir hata oluşursa ENO çıkışı lojik 0 değerini alır.

LADDER:



STL: **MOVW IN1, OUT**
MUL IN2,OUT

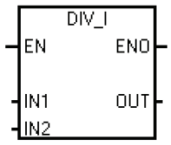
OPERANTLAR:
IN1,IN2: **IW, QW, VW, MW, SMW, SW, LW, AIW, AC, T, C, *VD, *LD, *AC, Sabit**
OUT: **ID, QD, VD, MD, SMD, SD, LD, AIW, AC, * VD, *LD, *AC**

Tam Sayı Bölme (DIV_I) Komutu

Bu komut IN1 girişinde bulunan word uzunluğundaki tam sayıyı, IN2 girişinde bulunan ve yine word uzunluğundaki tam sayıya böler. Bölüm sonucu OUT çıkışı ile belirtilen word uzunluğundaki hafıza alanına yazılır. Bölme işlemi sonucunda elde edilen kalan dikkate alınmaz. (herhangi bir alanda saklanmaz.)

Bölme işleminin sonucuna bağlı olarak SM1.0, SM1.1, SM1.2 ve SM1.3 özel hafıza bitleri etkilenir. Bölme işlemi sırasında bir hata oluşursa ENO çıkışı lojik 0 değerini alır.

LADDER:



STL: **MOVW IN1, OUT**
/ I IN2,OUT

OPERANTLAR:
IN1,IN2: **IW, QW, VW, MW, SMW, SW, LW, AIW, AC, T, C, *VD, *LD, *AC, Sabit**
OUT: **IW, QW, VW, MW, SMW, SW, LW, AC, T, C, * VD, *LD, *AC**

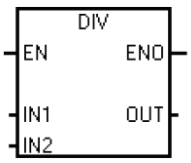
Double word uzunluğundaki tam sayıların bölünmesi için DIV_DI, double word uzunluğundaki alanda bulunan reel sayıların bölünmesi için ise DIV_R komutu kullanılır. Bu komutların kullanım şekli ve operantları için S7 – 200 kullanım kılavuzuna bakınız.

Kalanlı Tam Sayı Bölme (DIV) Komutu

Bu komut IN1 girişinde bulunan word uzunluğundaki tam sayıyı IN2 girişinde bulunan yine word uzunluğundaki tam sayıya böler. Bölme işlemi sonucunda elde edilen bölüm ve kalan OUT çıkışı ile belirtilen double word uzunluğundaki hafıza alanına yazılır. Bölüm, double word uzunluğundaki alanın düşük değerli wordüne, kalan ise yüksek değerli wordüne yerleşir.

Bölme işleminin sonucuna bağlı olarak SM1.0, SM1.1, SM1.2 ve SM1.3 özel hafıza bitleri etkilenir. Bölme işlemi sırasında bir hata oluşursa ENO çıkışı lojik 0 değerini alır.

LADDER:



STL:
MOVW IN1, OUT
DIV IN2,OUT

OPERANTLAR:
IN1,IN2: **IW, QW, VW, MW, SMW, SW, LW, AIW, AC, T, C, *VD, *LD, *AC, Sabit**
OUT: **ID, QD, VD, MD, SMD, SD, LD, AC, * VD, *LD, *AC**